

Data Structures and Algorithms

Laboratory Manual

Department of Robotics and Automation Engineering

Academic Year 2026

Table of Contents

1.	Array-Based Sensor Data Processing
2.	Stack Operations and Reversal of Actuator Commands
3.	Queue-Based Task Processing for Robot Movement Commands
4.	Singly Linked List for Robot Task IDs
5.	Doubly Linked List for Conveyor Stations
6.	Recursion: Factorial, Fibonacci, Binary Search and Tower of Hanoi
7.	Binary Search Tree for Component IDs
8.	Heap / Priority Queue for Emergency Alerts
9.	Linear Search vs Binary Search
10.	Performance Analysis of Sorting Algorithms
11.	Hashing with Linear Probing and Chaining for Machine IDs
12.	Graph Representation and BFS/DFS Traversal for Robot Movement

Array-Based Sensor Data Processing

AIM

Implement an array-based data processing program to store sensor readings and compute the number of readings, minimum value, maximum value, average value and range, using sample data from a temperature sensor.

THEORY

An array is a linear data structure that stores a fixed-size sequential collection of elements of the same type in contiguous memory locations. Arrays allow constant-time access to any element using its index, which makes them well suited for storing sensor readings collected over time, such as temperature, water-level, distance or motor-speed values. Once the readings are stored in an array, simple linear traversal techniques can be used to compute descriptive statistics: the count of readings, the minimum and maximum values (by comparing each element with a running minimum/maximum), the average (sum of all elements divided by the count), and the range (difference between the maximum and minimum values). Such statistical summaries are commonly used in embedded and automation systems to monitor sensor health and detect abnormal readings.

C PROGRAM

```
#include <stdio.h>

int main() {
    float readings[100];
    int n, i;
    float min, max, sum = 0, avg, range;

    printf("Enter number of temperature sensor readings: ");
    scanf("%d", &n);

    printf("Enter %d readings (in degree Celsius):\n", n);
    for (i = 0; i < n; i++) {
        scanf("%f", &readings[i]);
    }

    min = max = readings[0];
    for (i = 0; i < n; i++) {
        sum += readings[i];
        if (readings[i] < min)
            min = readings[i];
        if (readings[i] > max)
            max = readings[i];
    }

    avg = sum / n;
    range = max - min;

    printf("\n--- Sensor Data Report ---\n");
    printf("Number of readings : %d\n", n);
    printf("Minimum value      : %.2f\n", min);
    printf("Maximum value       : %.2f\n", max);
    printf("Average value        : %.2f\n", avg);
    printf("Range                : %.2f\n", range);

    return 0;
}
```

SAMPLE INPUT / OUTPUT

Sample Input:
Number of readings: 5
Readings: 28.5 30.2 26.8 31.0 29.4

Sample Output:
Number of readings : 5
Minimum value : 26.80
Maximum value : 31.00
Average value : 29.18
Range : 4.20

Stack Operations and Reversal of Actuator Commands

AIM

Design and demonstrate stack operations such as push, pop, peek and display, and apply the stack to reverse a sequence of actuator commands.

THEORY

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle, where insertion and deletion take place only from one end called the top. The primary operations on a stack are push (insert an element at the top), pop (remove and return the top element), peek/top (view the top element without removing it) and display (view all elements). Stacks are useful in automation systems for tasks such as reversing a sequence of actuator commands before execution, verifying the correct nesting/order of machine-operation steps (similar to balanced parentheses checking), and implementing undo functionality for robot motion sequences. In this program, an array-based stack is used to reverse a sequence of actuator command codes.

C PROGRAM

```
#include <stdio.h>
#define MAX 50

int stack[MAX];
int top = -1;

void push(int val) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = val;
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
        return -1;
    }
    return stack[top--];
}

int peek() {
    if (top == -1) {
        printf("Stack is empty\n");
        return -1;
    }
    return stack[top];
}

void display() {
    int i;
    if (top == -1) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack (top -> bottom): ");
    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

int main() {
    int choice, val, i, n;
    /* Actuator command codes: 1-Forward 2-Reverse 3-Left 4-Right 5-Stop */
    int commands[] = {1, 3, 2, 4, 5};
    n = 5;

    printf("Demonstration: Reversing actuator command sequence using a Stack\n");
    printf("Original command sequence: ");
    for (i = 0; i < n; i++)
        printf("%d ", commands[i]);
    printf("\n");

    for (i = 0; i < n; i++)
        push(commands[i]);

    printf("Reversed command sequence: ");
    while (top != -1)
        printf("%d ", pop());
    printf("\n");
```

```
printf("\nMenu-driven Stack Operations:\n");
do {
    printf("\n1.Push  2.Pop  3.Peek  4.Display  5.Exit\nEnter choice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1:
            printf("Enter value to push: ");
            scanf("%d", &val);
            push(val);
            break;
        case 2:
            val = pop();
            if (val != -1)
                printf("Popped value: %d\n", val);
            break;
        case 3:
            val = peek();
            if (val != -1)
                printf("Top value: %d\n", val);
            break;
        case 4:
            display();
            break;
        case 5:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice\n");
    }
} while (choice != 5);

return 0;
}
```

SAMPLE INPUT / OUTPUT

Sample Output:

Original command sequence: 1 3 2 4 5

Reversed command sequence: 5 4 2 3 1

Queue-Based Task Processing for Robot Movement Commands

AIM

Construct a queue-based task processing program to perform insertion, deletion and display operations for robot movement commands.

THEORY

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle, where insertion (enqueue) takes place at the rear end and deletion (dequeue) takes place at the front end. Queues are widely used in automation and robotics for buffering robot movement commands, scheduling workshop jobs, managing pump requests, and buffering sensor data so that tasks are processed in the exact order in which they arrive. This program implements a linear array-based queue with insert, delete and display operations to process a sequence of robot movement command codes.

C PROGRAM

```
#include <stdio.h>
#define MAX 50

int queue[MAX];
int front = -1, rear = -1;

void insertQueue(int val) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1)
        front = 0;
    queue[++rear] = val;
}

int deleteQueue() {
    int val;
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return -1;
    }
    val = queue[front++];
    return val;
}

void display() {
    int i;
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue (front -> rear): ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    int choice, val;

    printf("Task Processing Queue - Robot Movement Commands\n");
    printf("Codes: 1-Move Forward 2-Move Backward 3-Turn Left 4-Turn Right 5-Grip\n");

    do {
        printf("\n1.Insert  2.Delete  3.Display  4.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter command code to insert: ");
                scanf("%d", &val);
                insertQueue(val);
                break;
            case 2:
                val = deleteQueue();
                if (val != -1)
                    printf("Command processed (removed): %d\n", val);
                break;
            case 3:
                display();
                break;
            case 4:
```

```
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice\n");
    }
} while (choice != 4);

return 0;
}
```

SAMPLE INPUT / OUTPUT

Sample Interaction:

Insert -> 1, 3, 5 (Forward, Left, Grip)

Display -> Queue (front -> rear): 1 3 5

Delete -> Command processed (removed): 1

Singly Linked List for Robot Task IDs

AIM

Implement a singly linked list to perform insertion, deletion, searching and traversal operations for maintaining robot task IDs.

THEORY

A singly linked list is a linear data structure consisting of nodes, where each node contains a data field and a pointer to the next node in the sequence. Unlike arrays, linked lists allocate memory dynamically, allowing efficient insertion and deletion without shifting elements. Each node in this program stores a robot task ID. The list supports insertion of a new task ID at the end, deletion of a task ID, linear search for a given task ID, and traversal to display all task IDs currently maintained by the robot controller.

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int taskID;
    struct Node *next;
};

struct Node *head = NULL;

void insertEnd(int id) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    struct Node *temp;
    newNode->taskID = id;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}

void deleteNode(int id) {
    struct Node *temp = head, *prev = NULL;

    if (temp != NULL && temp->taskID == id) {
        head = temp->next;
        free(temp);
        printf("Task ID %d deleted\n", id);
        return;
    }

    while (temp != NULL && temp->taskID != id) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Task ID %d not found\n", id);
        return;
    }

    prev->next = temp->next;
    free(temp);
    printf("Task ID %d deleted\n", id);
}

void search(int id) {
    struct Node *temp = head;
    int pos = 1;
    while (temp != NULL) {
        if (temp->taskID == id) {
            printf("Task ID %d found at position %d\n", id, pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
}
```

```

        printf("Task ID %d not found\n", id);
    }

void traverse() {
    struct Node *temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Robot Task IDs: ");
    while (temp != NULL) {
        printf("%d -> ", temp->taskID);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, val;

    do {
        printf("\n1.Insert  2.Delete  3.Search  4.Traverse  5.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter Task ID to insert: ");
                scanf("%d", &val);
                insertEnd(val);
                break;
            case 2:
                printf("Enter Task ID to delete: ");
                scanf("%d", &val);
                deleteNode(val);
                break;
            case 3:
                printf("Enter Task ID to search: ");
                scanf("%d", &val);
                search(val);
                break;
            case 4:
                traverse();
                break;
            case 5:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice\n");
        }
    } while (choice != 5);

    return 0;
}

```

SAMPLE INPUT / OUTPUT

Sample Interaction:

Insert -> 101, 102, 103

Traverse -> Robot Task IDs: 101 -> 102 -> 103 -> NULL

Search 102 -> Task ID 102 found at position 2

Delete 101 -> Task ID 101 deleted

Doubly Linked List for Conveyor Stations

AIM

Model a doubly linked list to support insertion, deletion, forward traversal and reverse traversal for representing conveyor stations.

THEORY

A doubly linked list is a linear data structure in which each node contains a data field along with two pointers: one pointing to the previous node and one pointing to the next node. This bidirectional linking allows traversal of the list in both forward and reverse directions, which is useful for representing systems such as a conveyor belt with multiple stations, or a robot that can move both forward and reverse between checkpoints. This program stores conveyor station IDs and supports insertion, deletion, forward traversal and reverse traversal.

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int stationID;
    struct Node *prev;
    struct Node *next;
};

struct Node *head = NULL;

void insertEnd(int id) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    struct Node *temp;
    newNode->stationID = id;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}

void deleteNode(int id) {
    struct Node *temp = head;

    while (temp != NULL && temp->stationID != id)
        temp = temp->next;

    if (temp == NULL) {
        printf("Station %d not found\n", id);
        return;
    }

    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        head = temp->next;

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    free(temp);
    printf("Station %d removed\n", id);
}

void traverseForward() {
    struct Node *temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
    printf("Conveyor stations (forward): ");
    while (temp != NULL) {
        printf("%d ", temp->stationID);
        temp = temp->next;
    }
}
```

```

    }
    printf("\n");
}

void traverseReverse() {
    struct Node *temp = head;
    if (temp == NULL) {
        printf("List is empty\n");
        return;
    }
    while (temp->next != NULL)
        temp = temp->next;

    printf("Conveyor stations (reverse): ");
    while (temp != NULL) {
        printf("%d ", temp->stationID);
        temp = temp->prev;
    }
    printf("\n");
}

int main() {
    int choice, val;

    do {
        printf("\n1.Insert  2.Delete  3.Forward Traverse  4.Reverse Traverse  5.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter Station ID to insert: ");
                scanf("%d", &val);
                insertEnd(val);
                break;
            case 2:
                printf("Enter Station ID to delete: ");
                scanf("%d", &val);
                deleteNode(val);
                break;
            case 3:
                traverseForward();
                break;
            case 4:
                traverseReverse();
                break;
            case 5:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice\n");
        }
    } while (choice != 5);

    return 0;
}

```

SAMPLE INPUT / OUTPUT

Sample Interaction:

Insert -> 1, 2, 3, 4

Forward Traverse -> Conveyor stations (forward): 1 2 3 4

Reverse Traverse -> Conveyor stations (reverse): 4 3 2 1

Recursion: Factorial, Fibonacci, Binary Search and Tower of Hanoi

AIM

Apply recursion to solve selected computational problems such as factorial, Fibonacci series, binary search and Tower of Hanoi, and trace the recursive calls.

THEORY

Recursion is a programming technique in which a function calls itself, directly or indirectly, to solve a problem by breaking it down into smaller sub-problems of the same type. Every recursive function must have one or more base cases that stop the recursion, and a recursive case that reduces the problem size and moves it towards the base case. Factorial and Fibonacci are classic examples of recursion on numeric sequences; binary search uses recursion to repeatedly divide a sorted array in half; and the Tower of Hanoi problem uses recursion to move a stack of discs between pegs while obeying size constraints, illustrating how a complex sequence of engineering-like steps (analogous to repeated automation tasks) can be broken into smaller repeated actions.

C PROGRAM

```
#include <stdio.h>

long factorial(int n) {
    if (n <= 1)
        return 1;
    return n * factorial(n - 1);
}

int fibonacci(int n) {
    if (n <= 1)
        return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int binarySearch(int arr[], int low, int high, int key) {
    int mid;
    if (low > high)
        return -1;
    mid = (low + high) / 2;
    if (arr[mid] == key)
        return mid;
    else if (key < arr[mid])
        return binarySearch(arr, low, mid - 1, key);
    else
        return binarySearch(arr, mid + 1, high, key);
}

void towerOfHanoi(int n, char from, char aux, char to) {
    if (n == 0)
        return;
    towerOfHanoi(n - 1, from, to, aux);
    printf("Move disc %d from %c to %c\n", n, from, to);
    towerOfHanoi(n - 1, aux, from, to);
}

int main() {
    int choice, n, i, key, pos;
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 45};
    int size = 8;

    do {
        printf("\n--- Recursion Menu ---\n");
        printf("1.Factorial  2.Fibonacci Series  3.Binary Search  4.Tower of Hanoi  5.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter number: ");
                scanf("%d", &n);
                printf("Factorial of %d = %ld\n", n, factorial(n));
                break;
            case 2:
                printf("Enter number of terms: ");
                scanf("%d", &n);
                printf("Fibonacci Series: ");
                for (i = 0; i < n; i++)
                    printf("%d ", fibonacci(i));
                printf("\n");
                break;
            case 3:
                printf("Enter array: ");
                for (i = 0; i < size; i++)
                    scanf("%d", &arr[i]);
                printf("Enter key: ");
                scanf("%d", &key);
                pos = binarySearch(arr, 0, size - 1, key);
                if (pos != -1)
                    printf("Element found at index %d\n", pos);
                else
                    printf("Element not found\n");
                break;
            case 4:
                printf("Enter number of discs: ");
                scanf("%d", &n);
                printf("Enter source, auxiliary and destination pegs: ");
                scanf("%c%c%c", &from, &aux, &to);
                towerOfHanoi(n, from, aux, to);
                break;
            case 5:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice\n");
                break;
        }
    } while (choice != 5);
}
```

```

        break;
    case 3:
        printf("Sorted array (component codes): ");
        for (i = 0; i < size; i++)
            printf("%d ", arr[i]);
        printf("\nEnter key to search: ");
        scanf("%d", &key);
        pos = binarySearch(arr, 0, size - 1, key);
        if (pos != -1)
            printf("Element found at index %d\n", pos);
        else
            printf("Element not found\n");
        break;
    case 4:
        printf("Enter number of discs: ");
        scanf("%d", &n);
        printf("Steps to move %d discs from A to C using B as auxiliary:\n", n);
        towerOfHanoi(n, 'A', 'B', 'C');
        break;
    case 5:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice\n");
    }
} while (choice != 5);

return 0;
}

```

SAMPLE INPUT / OUTPUT

Sample Output (Tower of Hanoi, n = 2):

```

Move disc 1 from A to B
Move disc 2 from A to C
Move disc 1 from B to C

```

Binary Search Tree for Component IDs

AIM

Construct a binary search tree and perform insertion, searching, inorder traversal, preorder traversal and postorder traversal using component IDs.

THEORY

A Binary Search Tree (BST) is a hierarchical, non-linear data structure in which every node has at most two children, referred to as the left child and the right child. For every node, all values in its left subtree are smaller than the node's value, and all values in its right subtree are greater. This ordering property allows efficient insertion and searching in $O(\log n)$ time on average. A BST can be traversed in three common depth-first orders: inorder (left, root, right), which visits nodes in ascending order; preorder (root, left, right); and postorder (left, right, root). In this program, component IDs used in an automation cell are stored in a BST.

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int id;
    struct Node *left, *right;
};

struct Node *createNode(int id) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->id = id;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node *insert(struct Node *root, int id) {
    if (root == NULL)
        return createNode(id);

    if (id < root->id)
        root->left = insert(root->left, id);
    else if (id > root->id)
        root->right = insert(root->right, id);

    return root;
}

struct Node *search(struct Node *root, int id) {
    if (root == NULL || root->id == id)
        return root;

    if (id < root->id)
        return search(root->left, id);
    return search(root->right, id);
}

void inorder(struct Node *root) {
    if (root == NULL)
        return;
    inorder(root->left);
    printf("%d ", root->id);
    inorder(root->right);
}

void preorder(struct Node *root) {
    if (root == NULL)
        return;
    printf("%d ", root->id);
    preorder(root->left);
    preorder(root->right);
}

void postorder(struct Node *root) {
    if (root == NULL)
        return;
    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->id);
}

int main() {
```

```

struct Node *root = NULL;
int choice, val;

do {
    printf("\n--- BST Menu (Component IDs) ---\n");
    printf("1.Insert  2.Search  3.Inorder  4.Preorder  5.Postorder  6.Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter Component ID to insert: ");
            scanf("%d", &val);
            root = insert(root, val);
            break;
        case 2:
            printf("Enter Component ID to search: ");
            scanf("%d", &val);
            if (search(root, val) != NULL)
                printf("Component ID %d found\n", val);
            else
                printf("Component ID %d not found\n", val);
            break;
        case 3:
            printf("Inorder Traversal: ");
            inorder(root);
            printf("\n");
            break;
        case 4:
            printf("Preorder Traversal: ");
            preorder(root);
            printf("\n");
            break;
        case 5:
            printf("Postorder Traversal: ");
            postorder(root);
            printf("\n");
            break;
        case 6:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice\n");
    }
} while (choice != 6);

return 0;
}

```

SAMPLE INPUT / OUTPUT

Sample Interaction:

```

Insert -> 50, 30, 70, 20, 40
Inorder  -> 20 30 40 50 70
Preorder  -> 50 30 20 40 70
Postorder -> 20 40 30 70 50

```

Heap / Priority Queue for Emergency Alerts

AIM

Implement a heap or priority queue to arrange tasks according to priority, using emergency stop alerts and robot job priority as examples.

THEORY

A heap is a complete binary tree that satisfies the heap-order property: in a max-heap, every parent node's value is greater than or equal to the values of its children, so the maximum element is always at the root. A priority queue built on a max-heap allows the element with the highest priority to be retrieved first (extract-max), regardless of the order in which elements were inserted. This makes heaps ideal for automation systems where certain events, such as an emergency stop alert, must always be processed before lower-priority tasks such as routine maintenance requests. This program implements an array-based max-heap with insert and extract-max operations.

C PROGRAM

```
#include <stdio.h>
#define MAX 50

int heap[MAX];
int size = 0;

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void insert(int priority) {
    int i = size++;
    heap[i] = priority;

    while (i != 0 && heap[(i - 1) / 2] < heap[i]) {
        swap(&heap[i], &heap[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

void heapify(int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < size && heap[left] > heap[largest])
        largest = left;
    if (right < size && heap[right] > heap[largest])
        largest = right;

    if (largest != i) {
        swap(&heap[i], &heap[largest]);
        heapify(largest);
    }
}

int extractMax() {
    int root;
    if (size == 0) {
        printf("Priority queue is empty\n");
        return -1;
    }
    root = heap[0];
    heap[0] = heap[size - 1];
    size--;
    heapify(0);
    return root;
}

void display() {
    int i;
    if (size == 0) {
        printf("Priority queue is empty\n");
        return;
    }
    printf("Pending tasks (by priority value): ");
    for (i = 0; i < size; i++)
        printf("%d ", heap[i]);
    printf("\n");
}
```

```

}

int main() {
    int choice, val;

    printf("Max-Heap based Priority Queue - Emergency Stop / Robot Job Priority\n");
    printf("(Higher number = higher priority, e.g. 10 = Emergency Stop)\n");

    do {
        printf("\n1.Insert Task  2.Process Highest Priority  3.Display  4.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter task priority: ");
                scanf("%d", &val);
                insert(val);
                break;
            case 2:
                val = extractMax();
                if (val != -1)
                    printf("Processing task with priority: %d\n", val);
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice\n");
        }
    } while (choice != 4);

    return 0;
}

```

SAMPLE INPUT / OUTPUT

Sample Interaction:

Insert -> 3 (routine), 7 (maintenance), 10 (emergency stop), 5 (job)

Process Highest Priority -> Processing task with priority: 10

Process Highest Priority -> Processing task with priority: 7

Linear Search vs Binary Search

AIM

Evaluate linear search and binary search using a list of component numbers and compare the number of comparisons required by each method.

THEORY

Linear search examines each element of a list sequentially, one by one, until the required element is found or the list is exhausted; its worst-case time complexity is $O(n)$. Binary search operates only on a sorted array and repeatedly divides the search interval in half, comparing the key with the middle element, giving a much better worst-case time complexity of $O(\log n)$. This program searches for a given component ID in a sorted list of component numbers using both methods and reports the number of comparisons made by each, demonstrating the efficiency gained by binary search on sorted data.

C PROGRAM

```
#include <stdio.h>

int linearSearch(int arr[], int n, int key, int *comparisons) {
    int i;
    *comparisons = 0;
    for (i = 0; i < n; i++) {
        (*comparisons)++;
        if (arr[i] == key)
            return i;
    }
    return -1;
}

int binarySearch(int arr[], int n, int key, int *comparisons) {
    int low = 0, high = n - 1, mid;
    *comparisons = 0;
    while (low <= high) {
        mid = (low + high) / 2;
        (*comparisons)++;
        if (arr[mid] == key)
            return mid;
        else if (key < arr[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return -1;
}

int main() {
    int arr[] = {101, 105, 110, 115, 120, 125, 130, 135, 140, 145};
    int n = 10, i;
    int key, posLinear, posBinary, compLinear, compBinary;

    printf("Component ID list: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    printf("Enter Component ID to search: ");
    scanf("%d", &key);

    posLinear = linearSearch(arr, n, key, &compLinear);
    posBinary = binarySearch(arr, n, key, &compBinary);

    printf("\n--- Linear Search ---\n");
    if (posLinear != -1)
        printf("Found at index %d, Comparisons = %d\n", posLinear, compLinear);
    else
        printf("Not found, Comparisons = %d\n", compLinear);

    printf("\n--- Binary Search ---\n");
    if (posBinary != -1)
        printf("Found at index %d, Comparisons = %d\n", posBinary, compBinary);
    else
        printf("Not found, Comparisons = %d\n", compBinary);

    printf("\nConclusion: Binary search required fewer comparisons than linear\n");
    printf("search because the component ID list was sorted.\n");

    return 0;
}
```

SAMPLE INPUT / OUTPUT

```
Sample Output (key = 135):  
Linear Search  -> Found at index 7, Comparisons = 8  
Binary Search  -> Found at index 7, Comparisons = 2
```

Performance Analysis of Sorting Algorithms

AIM

Analyse the performance of sorting algorithms such as bubble sort, insertion sort and quick sort by sorting a list of motor speed values.

THEORY

Sorting algorithms arrange the elements of a list in a specific order, usually ascending or descending. Bubble sort repeatedly steps through the list, comparing adjacent elements and swapping them if they are in the wrong order ($O(n^2)$ worst case). Insertion sort builds the sorted list one element at a time by inserting each new element into its correct position among the already-sorted elements ($O(n^2)$ worst case, but efficient for nearly sorted data). Quick sort uses a divide-and-conquer strategy, selecting a pivot element and partitioning the array so that smaller elements come before the pivot and larger elements come after it, then recursively sorting the two partitions ($O(n \log n)$ average case). This program sorts a set of motor speed readings using all three algorithms and counts the number of comparisons and swaps made by each, allowing their performance to be compared.

C PROGRAM

```
#include <stdio.h>

void bubbleSort(int arr[], int n, long *comparisons, long *swaps) {
    int i, j, temp;
    *comparisons = 0;
    *swaps = 0;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            (*comparisons)++;
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                (*swaps)++;
            }
        }
    }
}

void insertionSort(int arr[], int n, long *comparisons, long *swaps) {
    int i, j, key;
    *comparisons = 0;
    *swaps = 0;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0) {
            (*comparisons)++;
            if (arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
                (*swaps)++;
            } else {
                break;
            }
        }
        arr[j + 1] = key;
    }
}

int partition(int arr[], int low, int high, long *comparisons, long *swaps) {
    int pivot = arr[high];
    int i = low - 1, j, temp;

    for (j = low; j < high; j++) {
        (*comparisons)++;
        if (arr[j] < pivot) {
            i++;
            temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
            (*swaps)++;
        }
    }
    temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
    (*swaps)++;
    return i + 1;
}

void quickSort(int arr[], int low, int high, long *comparisons, long *swaps) {
```

```

        if (low < high) {
            int pi = partition(arr, low, high, comparisons, swaps);
            quickSort(arr, low, pi - 1, comparisons, swaps);
            quickSort(arr, pi + 1, high, comparisons, swaps);
        }
    }

void printArray(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int original[] = {75, 42, 88, 30, 95, 60, 15, 50};
    int n = 8, i;
    int arr1[8], arr2[8], arr3[8];
    long comp = 0, swp = 0;

    for (i = 0; i < n; i++) {
        arr1[i] = original[i];
        arr2[i] = original[i];
        arr3[i] = original[i];
    }

    printf("Original motor speed values: ");
    printArray(original, n);

    bubbleSort(arr1, n, &comp, &swp);
    printf("\nBubble Sort    : ");
    printArray(arr1, n);
    printf("Comparisons = %ld, Swaps = %ld\n", comp, swp);

    insertionSort(arr2, n, &comp, &swp);
    printf("\nInsertion Sort: ");
    printArray(arr2, n);
    printf("Comparisons = %ld, Swaps = %ld\n", comp, swp);

    comp = 0; swp = 0;
    quickSort(arr3, 0, n - 1, &comp, &swp);
    printf("\nQuick Sort      : ");
    printArray(arr3, n);
    printf("Comparisons = %ld, Swaps = %ld\n", comp, swp);

    return 0;
}

```

SAMPLE INPUT / OUTPUT

Sample Output:

Original motor speed values: 75 42 88 30 95 60 15 50

Bubble Sort : 15 30 42 50 60 75 88 95 (Comparisons = 28, Swaps = 16)

Insertion Sort : 15 30 42 50 60 75 88 95 (Comparisons and Swaps vary by input order)

Quick Sort : 15 30 42 50 60 75 88 95 (Comparisons and Swaps vary by pivot choice)

Hashing with Linear Probing and Chaining for Machine IDs

AIM

Implement hashing using linear probing and chaining to store and retrieve machine IDs, and demonstrate collision handling.

THEORY

Hashing is a technique that maps a key to a specific index of a hash table using a hash function, typically $\text{key} \% \text{table_size}$, enabling average-case $O(1)$ insertion and search. When two different keys map to the same index, a collision occurs, and it must be resolved. Linear probing resolves a collision by searching sequentially for the next empty slot in the table. Chaining resolves a collision by storing all keys that map to the same index in a linked list attached to that index, so multiple keys can share one slot without overwriting each other. This program stores machine IDs using both linear probing and separate chaining and allows insertion, searching and display for each technique.

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#define TABLE_SIZE 10

/* ----- Linear Probing ----- */
int hashTableLP[TABLE_SIZE];
int occupied[TABLE_SIZE];

void initLP() {
    int i;
    for (i = 0; i < TABLE_SIZE; i++) {
        hashTableLP[i] = -1;
        occupied[i] = 0;
    }
}

void insertLP(int key) {
    int idx = key % TABLE_SIZE;
    int start = idx;

    while (occupied[idx]) {
        idx = (idx + 1) % TABLE_SIZE;
        if (idx == start) {
            printf("Hash table full, cannot insert %d\n", key);
            return;
        }
    }
    hashTableLP[idx] = key;
    occupied[idx] = 1;
    printf("Machine ID %d inserted at index %d\n", key, idx);
}

void searchLP(int key) {
    int idx = key % TABLE_SIZE;
    int start = idx;

    while (occupied[idx]) {
        if (hashTableLP[idx] == key) {
            printf("Machine ID %d found at index %d\n", key, idx);
            return;
        }
        idx = (idx + 1) % TABLE_SIZE;
        if (idx == start)
            break;
    }
    printf("Machine ID %d not found\n", key);
}

void displayLP() {
    int i;
    printf("\nHash Table (Linear Probing):\n");
    for (i = 0; i < TABLE_SIZE; i++) {
        if (occupied[i])
            printf("Index %d : %d\n", i, hashTableLP[i]);
        else
            printf("Index %d : empty\n", i);
    }
}

/* ----- Chaining ----- */
struct ChainNode {
```

```

        int key;
        struct ChainNode *next;
    };
};

struct ChainNode *hashTableChain[TABLE_SIZE];

void initChain() {
    int i;
    for (i = 0; i < TABLE_SIZE; i++)
        hashTableChain[i] = NULL;
}

void insertChain(int key) {
    int idx = key % TABLE_SIZE;
    struct ChainNode *newNode = (struct ChainNode *)malloc(sizeof(struct ChainNode));
    newNode->key = key;
    newNode->next = hashTableChain[idx];
    hashTableChain[idx] = newNode;
    printf("Machine ID %d inserted at index %d (chaining)\n", key, idx);
}

void searchChain(int key) {
    int idx = key % TABLE_SIZE;
    struct ChainNode *temp = hashTableChain[idx];

    while (temp != NULL) {
        if (temp->key == key) {
            printf("Machine ID %d found at index %d\n", key, idx);
            return;
        }
        temp = temp->next;
    }
    printf("Machine ID %d not found\n", key);
}

void displayChain() {
    int i;
    struct ChainNode *temp;
    printf("\nHash Table (Chaining):\n");
    for (i = 0; i < TABLE_SIZE; i++) {
        printf("Index %d : ", i);
        temp = hashTableChain[i];
        while (temp != NULL) {
            printf("%d -> ", temp->key);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}

int main() {
    int choice, val;

    initLP();
    initChain();

    do {
        printf("\n--- Hashing Menu (Machine IDs) ---\n");
        printf("1.Insert (Linear Probing) 2.Search (Linear Probing) 3.Display (Linear Probing)\n");
        printf("4.Insert (Chaining) 5.Search (Chaining) 6.Display (Chaining) 7.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter Machine ID: ");
                scanf("%d", &val);
                insertLP(val);
                break;
            case 2:
                printf("Enter Machine ID to search: ");
                scanf("%d", &val);
                searchLP(val);
                break;
            case 3:
                displayLP();
                break;
            case 4:
                printf("Enter Machine ID: ");
                scanf("%d", &val);
                insertChain(val);
                break;
            case 5:
                printf("Enter Machine ID to search: ");
                scanf("%d", &val);
                searchChain(val);
                break;
            case 6:
                displayChain();

```

```
        break;
    case 7:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice\n");
    }
} while (choice != 7);

return 0;
}
```

SAMPLE INPUT / OUTPUT

Sample Interaction (Linear Probing, Table Size = 10):

Insert 15 -> Machine ID 15 inserted at index 5

Insert 25 -> Machine ID 25 inserted at index 6 (collision at index 5, probed forward)

Search 25 -> Machine ID 25 found at index 6

Graph Representation and BFS/DFS Traversal for Robot Movement

AIM

Model a simple path or network using graph representation and perform BFS and DFS traversal for robot movement between locations.

THEORY

A graph is a non-linear data structure consisting of a set of vertices (nodes) and a set of edges connecting pairs of vertices. In this program, an adjacency matrix is used to represent connectivity between locations in a workshop or warehouse layout, where each vertex represents a location and each edge represents a valid path a robot can travel. Breadth-First Search (BFS) explores the graph level by level using a queue, visiting all neighbours of a vertex before moving to the next level, and is useful for finding the shortest path in terms of number of hops. Depth-First Search (DFS) explores as far as possible along each branch using recursion (or a stack) before backtracking, and is useful for checking connectivity and exploring all reachable locations from a starting point.

C PROGRAM

```
#include <stdio.h>
#define MAX 10

int adj[MAX][MAX];
int n;

void addEdge(int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1;
}

void BFS(int start) {
    int queue[MAX], front = 0, rear = 0, i, u;
    int visitedLocal[MAX] = {0};

    visitedLocal[start] = 1;
    queue[rear++] = start;

    printf("BFS traversal from location %d: ", start);
    while (front < rear) {
        u = queue[front++];
        printf("%d ", u);
        for (i = 0; i < n; i++) {
            if (adj[u][i] == 1 && !visitedLocal[i]) {
                visitedLocal[i] = 1;
                queue[rear++] = i;
            }
        }
    }
    printf("\n");
}

void DFSUtil(int u, int visitedLocal[]) {
    int i;
    visitedLocal[u] = 1;
    printf("%d ", u);
    for (i = 0; i < n; i++) {
        if (adj[u][i] == 1 && !visitedLocal[i])
            DFSUtil(i, visitedLocal);
    }
}

void DFS(int start) {
    int visitedLocal[MAX] = {0};
    printf("DFS traversal from location %d: ", start);
    DFSUtil(start, visitedLocal);
    printf("\n");
}

int main() {
    int edges, u, v, start, i, j;

    printf("Enter number of locations (nodes) in the workshop/warehouse layout: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            adj[i][j] = 0;
```

```
printf("Enter number of connections (edges/paths): ");
scanf("%d", &edges);

printf("Enter each connection as: location1 location2\n");
for (i = 0; i < edges; i++) {
    scanf("%d %d", &u, &v);
    addEdge(u, v);
}

printf("Enter starting location for traversal: ");
scanf("%d", &start);

BFS(start);
DFS(start);

return 0;
}
```

SAMPLE INPUT / OUTPUT

Sample Input:
Locations: 5
Connections: 4 -> 0-1, 0-2, 1-3, 2-4
Start location: 0

Sample Output:
BFS traversal from location 0: 0 1 2 3 4
DFS traversal from location 0: 0 1 3 2 4

--- End of Manual ---